# Migration from Deprecated API in Java

Roman Štrobl
Faculty of Information Technology
Czech Technical University in Prague
Czech Republic
stroblr@fit.cvut.cz

Zdeněk Troníček
Faculty of Information Technology
Czech Technical University in Prague
Czech Republic
tronicek@fit.cvut.cz

## Abstract

When software components evolve, they change interfaces. Members that are obsolete are marked as deprecated and new members are added. We deal with the problem of migration from deprecated members to their replacement. We implemented two tools: Java Source Code Update Tool, which updates the source code based on a configuration file, and a generator, which heuristically figures out how to migrate from deprecated members and generates the configuration file. We evaluated these tools on five open source projects and the results are very encouraging.

***Categories and Subject Descriptors*** D.2.3 [Software Engineering]: Coding Tools and Techniques; D.2.7 [Software Engineering]: Distribution, Maintenance, and Enhancement

***Keywords*** Component upgrade, software evolution, Java, deprecated API, migration

## 1. Introduction

We deal with a problem related to the evolution of Java component-based software. When a component evolves, it may change its interface (Application Programming Interface, API). For instance, a class can be renamed or a method can change its signature. Since these changes can break component clients, they are usually introduced gradually: first, obsolete API members are deprecated and new API members are introduced, and then, after some time, these deprecated members are removed. When an API member is deprecated, clients are expected to migrate to its replacement. In this paper, we address the problem of migration from deprecated API members to non-deprecated ones.


*SPLASH '13*, October 26–31, 2013, Indianapolis, Indiana, USA.
ACM 978-1-4503-1995-9/13/10.
http://dx.doi.org/10.1145/2508075.2508093

## 2. JaSCUT

In order to facilitate migration, we implemented Java Source Code Update Tool (JaSCUT) [1], which is able to transform the Java source code based on a configuration file. For instance, a change from `enable()` to `setEnabled(true)` in `java.awt.Component` is described in the configuration file as follows:

```
<change-method-signature>
  <type>java.awt.Component</type>
  <method-orig>enable</method-orig>
  <args-orig/>
  <method-new>setEnabled</method-new>
  <args-new>
    <arg>
      <name>b</name>
      <type>boolean</type>
      <value kind="literal">true</value>
    </arg>
  </args-new>
</change-method-signature>
```

The following refactorings are supported in JaSCUT: rename method, move static method, change method signature, rename type, move type, rename field, move static field, replace constructor call with a factory method call, replace a field reference with a method call, and combinations of these refactorings.

## 3. Generator

Since preparing the configuration file for JaSCUT manually can be tedious, we implemented a desktop application that parses component source code and heuristically searches for replacements of deprecated API members. We use the following heuristics:

- Deprecated Javadoc heuristic
- Delegation heuristic
- Header similarity heuristic
- Type pairing heuristic
- Transitive deprecation heuristic

The Deprecated Javadoc heuristic analyzes the text in the @deprecated Javadoc tag and searches for known patterns such as "replaced by {@link …}". If such pattern is found,

the heuristic extracts the value specified in the link, and stores it as a replacement of the deprecated API member. We use the patterns as follows:

1. replaced by [the] {@link ...}
2. use [the] {@link ...}
3. see [the] {@link ...}
4. access [the] {@link ...}
5. {@link ...} (only at the beginning of the @deprecated tag)
6. replaced by [the] <code>...</code>
7. use [the] <code>...</code>
8. use [the] <a href='...'>...</a>
9. replaced by [free text]
10. use [free text]

The Delegation heuristic looks into the body of a deprecated method and if the body contains a single method call, the called method is considered as a replacement for the deprecated method. A similar approach is used for fields.

The Header similarity heuristic detects small changes in method and field names. We use the Damerau-Levenshtein string distance to measure the similarity of strings. The Type pairing heuristic creates pairs from members of renamed types. For example, if A is deprecated and replaced with B, the members of A are paired with members of B. The Transitive deprecation heuristic is used when a replacement is also deprecated. The heuristic finds the first non-deprecated member in a chain of replacements.

All these heuristics are combined and their results are merged based on weights. If the replacement found by the generator is uncertain, the rule is marked for review and the user is expected to either confirm or edit the rule.

# 4. Evaluation

We used five open source projects for the evaluation: Apache Http Client, OpenJDK, Joda Time, Bouncy Castle, and Apache PDFBox. In each project, we identified deprecated members and manually searched for their replacements. In case a hint was present in the Javadoc which said how to migrate from the deprecated member, we considered this migration rule as manually resolvable. We distinguished between regular rules and pairing rules. Regular rules are related to members that have either the @deprecated Javadoc tag or the @Deprecated annotation. Pairing rules are used for members that are not explicitly deprecated but they are inside of a deprecated type, and thus they must be migrated as well. Then we let the generator generate the migration rules, and we manually checked whether they are correct. We classified each rule in the following way: true positive (TP) if the API member can be resolved and the generator provides the same rule as manual resolution, true negative (TN) if the API member cannot be resolved and the generator marks the member as unresolvable, false negative (FN) if the API member can be resolved manually but the generator marks the member as unresolvable, and false positive (FP) if the generator provides incorrect resolution. Then we used this classification to count accuracy, precision, and recall as follows: accuracy = (TP + TN)/(TP + TN + FP + FN), precision = TP/(TP + FP), recall = TP/(TP + FN).

**Table 1: Evaluation of automated migration**

| Project | Accuracy | Precision | Recall |
|---|---|---|---|
| Apache Http Client | 86% | 95% | 74% |
| OpenJDK | 83% | 92% | 66% |
| Joda Time | 97% | 100% | 78% |
| Bouncy Castle | 69% | 100% | 49% |
| Apache PDFBox | 79% | 100% | 71% |

# 5. Related Work

Perkins [3] proposed a technique, called *method inlining*, for refactoring client code in response to changes in library API. The technique is based on a simple idea: calls to deprecated methods are replaced by their bodies.

We are not aware of any tool developed specifically for migration from deprecated API, but the problem is similar to the problem of migration to an alternative API, which was investigated by several researchers. Henkel and Diwan [4] described the CatchUp! tool, which records refactorings when the component evolves and enables to replay them later. The same idea was implemented in the Eclipse IDE [5]. It could presumably be used for migration from deprecated API provided that we prepare the configuration file manually; however, its use in the Eclipse IDE is limited only to migration of a JAR file.

# 6. Conclusion

We evaluated the possibility of automated migration from deprecated API members to their replacements. The evaluation shows that the migration can be partially automated for most Java APIs.

# References


[1] JaSCUT. http://java.net/projects/jascut

[2] JaSCUT Config Generator. http://java.net/projects/jascutconf

[3] J. H. Perkins. Automatically generating refactorings to support API evolution, *ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*, pp. 111–114, 2005.

[4] J. Henkel and A. Diwan. CatchUp!: capturing and replaying refactorings to support API evolution, *International Conference on Software Engineering*, pp. 274–283, 2005.

[5] Eclipse IDE. http://www.eclipse.org

[6] R. Štrobl. *Generator of a configuration file for JaSCUT: Master's thesis.* Czech Technical University in Prague, Faculty of Information Technology, 2013.